

few partitions.

Goldilocks Version 4: Reduce to Distinct on Each Partition

Rather than trying to partition those pairs differently, we realized that the last four steps of the algorithm could be modified to work on input of tuples of `((cell value, column index), count)`. Then, in the first step, rather than mapping the records on each partition to `(cell value, column index)` pairs, we could map each partition to distinct pairs, keeping track of the number of times each `((cell value, column index))` pair appeared on that partition. By mapping to `distinct` on each partition, we reduce the number of duplicate keys without incurring a shuffle (as would a distributed `distinct` call on the entire RDD).

After creating these `((value, columnIndex), count)` tuples, we know that although we might have some repeated keys, we introduce a theoretical limit on the number of duplicate keys. Specifically, if the same value is present in a column on each partition, the maximum number of duplicate keys is (the number of columns * the number of partitions).⁴ Not only does this input step balance the data so it can be partitioned more effectively, it also dramatically reduces the total number of records to be shuffled. In the Goldilocks case, where 25% of the rows were zero, we now have roughly 75% the number of tuples to sort as the previous iteration of our solution.

AGGREGATE TO ((CELL VALUE, COLUMN INDEX), COUNT) ON EACH PARTITION

Example 6-21 is the code for this first step. Rather than mapping to

(cell_value, column_index) pairs, we mapped to ((cell value, column index), count) on each partition. We were transforming the columnar data into pairs by using a mapPartitions step. Now, rather than doing a flatMap of each row in the original data, we update a HashMap whose keys are the (value, index) pairs and whose values are the number of times that (cell value, column index) pair appears in that partition. Even if each column on a given partition has all unique values, we expect that this solution will not cause out-of-memory errors. We are making the HashMap on each partition, and so the size of the HashMap should still be smaller than the iterator of all the records. However, if we did have all distinct values, this might not be the case, because a hash map is a much less memory-efficient data structure than an iterator.

Example 6-21. Goldilocks version 4, aggregate on each partition

```
def getAggregatedValueColumnPairs(dataFrame: DataFrame):  
    RDD[((Double, Int), Long)] = {  
  
        val aggregatedValueColumnRDD =  
dataFrame.rdd.mapPartitions(rows => {  
            val valueColumnMap = new mutable.HashMap[(Double, Int),  
Long]()  
            rows.foreach(row => {  
                row.toSeq.zipWithIndex.foreach{ case (value, columnIndex)  
=>  
                    val key = (value.toString.toDouble, columnIndex)  
                    val count = valueColumnMap.getOrElseUpdate(key, 0)  
                    valueColumnMap.update(key, count + 1)  
                }  
            })  
  
            valueColumnMap.toIterator  
        })  
  
        aggregatedValueColumnRDD  
    }
```

SORT AND FIND RANK STATISTICS

The rest of the function is similar to the original version. We simply adjust for keeping track of the number of times the pair appears rather than assuming that each pair in the sorted RDD occurred once. The updated code is shown in Examples 6-22, 6-23, 6-24, and 6-25, with Example 6-26 bringing everything together.

Example 6-22. Goldilocks version 4, count values per column on each partition

```
private def getColumnsFreqPerPartition(
  sortedAggregatedValueColumnPairs: RDD[((Double, Int), Long)],
  numColumns : Int): Array[(Int, Array[Long])] = {

  val zero = Array.fill[Long](numColumns)(0)

  def aggregateColumnFrequencies(
    partitionIndex : Int, pairs : Iterator[((Double, Int),
Long))] = {
    val columnsFreq : Array[Long] = pairs.aggregate(zero)(
      (a : Array[Long], v : ((Double, Int), Long)) => {
        val ((value, colIndex), count) = v
        a(colIndex) = a(colIndex) + count
        a},
      (a : Array[Long], b : Array[Long]) => {
        a.zip(b).map{ case(aVal, bVal) => aVal + bVal}
      })

    Iterator((partitionIndex, columnsFreq))
  }

  sortedAggregatedValueColumnPairs.mapPartitionsWithIndex(
    aggregateColumnFrequencies).collect()
}
```

Example 6-23. Goldilocks version 4, determine locations of rank statistics on each partition

```
private def getRanksLocationsWithinEachPart(targetRanks :
List[Long],
  partitionColumnsFreq : Array[(Int, Array[Long])],
  numColumns : Int) : Array[(Int, List[(Int, Long)])] =
{

  val runningTotal = Array.fill[Long](numColumns)(0)
```

```

    partitionColumnsFreq.sortBy(_._1).map { case (partitionIndex,
columnsFreq)=>
    val relevantIndexList = new mutable.MutableList[(Int, Long)]
    ()

    columnsFreq.zipWithIndex.foreach{ case (colCount, colIndex)
=>
    val runningTotalCol = runningTotal(colIndex)

    val ranksHere: List[Long] = targetRanks.filter(rank =>
    runningTotalCol < rank && runningTotalCol + colCount >=
rank)

    relevantIndexList ++= ranksHere.map(
    rank => (colIndex, rank - runningTotalCol))

    runningTotal(colIndex) += colCount
    }

    (partitionIndex, relevantIndexList.toList)
  }
}

```

Example 6-24. Goldilocks version 4, filter for rank statistics

```

private def findTargetRanksIteratively(
    sortedAggregatedValueColumnPairs : RDD[(Double, Int),
Long]),
    ranksLocations : Array[(Int, List[(Int, Long)])]):
RDD[(Int, Double)] = {

    sortedAggregatedValueColumnPairs.mapPartitionsWithIndex((partition
Index : Int,
    aggregatedValueColumnPairs : Iterator[(Double, Int),
Long])) => {

        val targetsInThisPart: List[(Int, Long)] =
ranksLocations(partitionIndex)._2
        if (targetsInThisPart.nonEmpty) {
            FindTargetsSubRoutine.asIteratorToIteratorTransformation(
                aggregatedValueColumnPairs,
                targetsInThisPart)
        } else {
            Iterator.empty
        }
    })
}

```

Example 6-25. Goldilocks version 4, iterator-to-iterator transformation to filter for the rank statistics

```
def asIteratorToIteratorTransformation(
  valueColumnPairsIter : Iterator[((Double, Int), Long)],
  targetsInThisPart: List[(Int, Long)] ): Iterator[(Int,
Double)] = {

  val columnsRelativeIndex =
targetsInThisPart.groupBy(_._1).mapValues(_._2.map(_._2))
  val columnsInThisPart = targetsInThisPart.map(_._1).distinct

  val runningTotals : mutable.HashMap[Int, Long]= new
mutable.HashMap()
  runningTotals += columnsInThisPart.map(columnIndex =>
(columnIndex, 0L)).toMap

  //filter out the pairs that don't have a column index that is
in this part
  val pairsWithRanksInThisPart = valueColumnPairsIter.filter{
    case (((value, colIndex), count)) =>
      columnsInThisPart contains colIndex
  }

  // map the valueColumn pairs to a list of (colIndex, value)
pairs that correspond
  // to one of the desired rank statistics on this partition.
  pairsWithRanksInThisPart.flatMap{

    case (((value, colIndex), count)) =>

      val total = runningTotals(colIndex)
      val ranksPresent: List[Long] =
columnsRelativeIndex(colIndex)
                                                                    .filter(index => (index
<= count + total)
                                                                    && (index > total))

      val nextElems: Iterator[(Int, Double)] =
        ranksPresent.map(r => (colIndex, value)).toIterator

      //update the running totals
      runningTotals.update(colIndex, total + count)
      nextElems
    }
  }
}
```

Putting all the code together, we have a final solution to the Goldilocks

problem (Example 6-26).

Example 6-26. Goldilocks with hash map

```
def findRankStatistics(dataFrame: DataFrame, targetRanks:
List[Long]):
    Map[Int, Iterable[Double]] = {

        val aggregatedValueColumnPairs: RDD[((Double, Int), Long)] =
            getAggregatedValueColumnPairs(dataFrame)
        val sortedAggregatedValueColumnPairs =
            aggregatedValueColumnPairs.sortByKey()

        sortedAggregatedValueColumnPairs.persist(StorageLevel.MEMORY_AND_DISK)

        val numColumns = dataFrame.schema.length
        val partitionColumnsFreq =
            getColumnsFreqPerPartition(sortedAggregatedValueColumnPairs,
            numColumns)
        val ranksLocations =
            getRanksLocationsWithinEachPart(targetRanks,
            partitionColumnsFreq, numColumns)

        val targetRanksValues =
            findTargetRanksIteratively(sortedAggregatedValueColumnPairs,
            ranksLocations)
        targetRanksValues.groupByKey().collectAsMap()
    }
```

As expected, this solution led to a 4× to 5× speed up of an empty cluster.

GOLDILOCKS POSTMORTEM

Given the task of finding an arbitrary number of rank statistics by group (column), we have presented five solutions:

- “Goldilocks Version 0: Iterative Solution”. Our first solution iteratively looped through each group and performed a distributed sort, resulting in one stage and one expensive distributed sort per group.
- “Goldilocks Version 1: groupByKey Solution”. The next solution used groupByKey shuffle records associated with the same group onto the

same partition. Then we were able to sort each group in one stage by using `mapPartitions` to sort the values in each group.

- “Goldilocks Version 2: Secondary Sort”. Using the secondary sort technique, we improved our `groupByKey` solution by replacing the `groupByKey` and sorting steps with the `repartitionAndSortWithinPartitions` function to push the work of sorting each group into the shuffle stage.
- “Goldilocks Version 3: Sort on Cell Values”. Next, we realized that it was possible to solve the problem using only one sort on the value of each record, rather than the group. We developed a solution that keyed the records by value (rather than by group/column index), sorted all the records, and then performed a series of narrow transformations to collect the results. We expected the new sorting keys (the values in the columns) to contain fewer duplicates than the the size of each group, which we used as a key in version 2.
- “Goldilocks Version 4: Reduce to Distinct on Each Partition”. Finally, upon realizing that we had a high number of duplicate records in each group, we modified the previous solution to perform a map-side reduction before sorting the data. This solution had better results with our client’s skewed data.

We found that performance was dependent mostly on three characteristics of the input data: 1) the number of original records, 2) the number of groups (columns in this case) to compute the metrics on, and 3) the percentage of duplicate records by group. Note that these metrics are all relative to the size and shape of the hardware you are computing on. A small number of records really means that the records fit comfortably in the computational memory of all of the executors.

“Goldilocks Version 4: Reduce to Distinct on Each Partition” is not the best solution for all input data. The only one of these five solutions that is always “bad” is the `groupByKey` solution (version 1). We would expect

that version 1 should perform worse than the `repartitionAndSortWithinPartitions` (version 2) solution in all cases due to the limitations of `groupByKey` and the advantages of secondary sort (see [“Secondary Sort and repartitionAndSortWithinPartitions”](#)). The remaining four versions are all desirable in some cases.

Version 0 uses the value in each group to sort (as do solutions 3 and 4). Thus it is optimal there is only one group, because it doesn't require extra passes through the data.

Version 2 partitions by group. Thus it is optimal if there are many groups, and each group fits easily in memory on one executor. If the groups are sufficiently small, it would outperform versions 3 and 4, which sort on value and require three narrow transformations to determine the correct results, because this solution requires fewer passes through the data. With enough groups and rank statistics this solution may also be more likely to succeed than versions 3 and 4, both of which require keeping a hash map of rank statistics for each group in memory.

Version 3 partitions by values across all groups. Thus it is optimal if each group is very large, and there are few duplicate values across the columns. Unlike version 2, only the records with duplicate values need to be on one partition. Assuming that the number of duplicate values is less than the maximum group size this will lead to better parallelization.

Version 4 uses a narrow transformation to aggregate distinct values by group on the input partitions, then partitions by distinct values (like solution 3). Solution 4 is better than 3 only when there are many duplicate values in the columns. However, if all values are unique, this operation

provides no benefit and risks causing memory errors by creating a hash map of all the values on each partition.

In our use case, we had input data containing several thousand groups with 300 million records each. In each column (group) we saw that about 25% of records were identical. Versions 0, 1, and 2 did not complete on a 10-node, crowded cluster. Version 3 did complete, but version 4 led to a 4× improvement in production over version 3.

Conclusion

In this chapter, we have seen how to use functions in the `PairRDDFunctions` and `OrderedRDDFunctions` classes in ways that are more likely to succeed at scale. Much of the chapter has focused on specific techniques for working with wide transformations. We focused on some of the causes of memory errors during the shuffle stage such as avoiding aggregation operations like `groupByKey` that do not reduce the space needed to store all the records associated with each key. We have learned about partitioning: how thinking ahead toward the next key/value transformation and doing smart partitioning can reduce the number of shuffles we need. We have focused on some strategies to do fewer shuffles: using smart partitioning, maintaining partition information with narrow transformations, and leveraging co-location for joins. When shuffles are required, we have covered some techniques for reducing the cost of those shuffles. In particular, we have shown that unbalanced data, particularly a high number of duplicate values per key, is likely to slow down shuffles and cause memory errors.

We hope that in addition to presenting some tricks for improving the performance of RDD transformations, this chapter provides some tools for

reconceptualizing problem solving with Spark. By focusing on the Goldilocks use case, we have tried to show that writing performant Spark code sometimes requires a new kind of thinking. Simply stringing together the set of API calls that most obviously describe what you are trying to do often leads to a solution such as the one presented in “Goldilocks Version 1: groupByKey Solution”: an implementation that succeeds on relatively small, clean inputs and fails in the wild west of real-world data. Instead, we hope to show that despite the relative simplicity of writing distributed code with Spark, the best results can be achieved only with a close eye to how those distributed computations will be performed. We hope to encourage you to think of keys in Spark not as the index or category for the records, but as the axis of parallelization for the computational load. We encourage you wherever possible to design Spark routines with as much knowledge as possible about the size, distribution, and complexity of the data they are processing. Philosophically, we want to demonstrate how high-performance code often requires big rather than small changes—and that the most performant code is not always the cleanest.

¹ As we hope to demonstrate in this chapter, even in cases where out-of-the-box functionality in Spark SQL may cover your use case, the creative control offered by Spark Core can often allow us to develop a better routine for a very specific use case. Indeed the final solution we present in this chapter is able to do some clever reductions that probably make it faster than even a well-implemented rank statistics implementation in Spark SQL.

² Iterators can only be traversed once. Thus, iterators cannot be used without conversion because sorting requires multiple passes through the data. They must be converted to another data structure first. For more

about the iterator type, and its advantages and limitations, see [“Iterator-to-Iterator Transformations with mapPartitions”](#).

³ Often RDDs without known partitioners can be RDDs loaded from storage. In this case the RDD’s data is most often effectively partitioned in the same way as the underlying storage (e.g., the Splits in Hadoop). However, once the data is read into Spark, Spark does not know what the underlying partitioning is and consequently cannot take advantage of this information. This is one way in which checkpointing is different from simply saving an RDD to stable storage and then reading it manually. In the checkpointing case, Spark saves some metadata about the RDD including, if applicable, its partitioner.

⁴ Using a distributed `reduceByKey` could have reduced this to simply the number of columns at the cost of a shuffle.